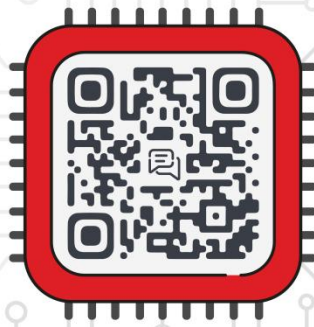




نظري

8



22



2640 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Spring Boot MVC

rest

د. غسان شذود



RB Informatics; 22/12/2026

محتوى مجاني غير مخصص للبيع التجاري

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

- سنتعرف في هذه المحاضرة كيفية بناء تطبيقات الويب والخدمات باستخدام springBoot
- أهم المواضيع التي سنتعرف عليها:
Spring MVC, REST, API
لنبدأ بمثال عن تطبيق Spring :

Curl-Client request examples

🔑 فائدة أريسيزية

تستخدم أداة Curl في سطر الأوامر لإرسال طلبات إلى الخادم واختبار واجهات البرمجة.

مثال 1:

```
1)curl GET -X http ://localhost :8080/books
2)curl -X POST http://localhost:8080/books \
-H "Content-Type: application/json" \
-d '{ "title": "The Catcher in the Rye", "author": "J.D.
Salinger", "publishedDate": "1951-07-16" }'
```

شرح المثال:

الأمر 1: هنا يطلب العميل قائمة بجميع الكتب من المسار /books /المفتوح على الجهاز المحلي عبر المنفذ 8080

الأمر 2: هنا لدينا post أي يجب إرسال معلومات للخادم حيث:

-H يمثل ال header

```

library-mvc/
├── src/main/java/com/library/ .....1
│   ├── LibraryMvcApplication.java    1.1
│   ├── entity/Book.java              1.2
│   ├── repository/BookRepository.java 1.3
│   ├── service/BookService.java       1.4
│   └── controller/BookController.java  1.5
├── src/main/resources/ .....2
│   ├── templates/books/              2.1
│   │   ├── list.html
│   │   ├── detail.html
│   │   ├── add-form.html
│   │   └── edit-form.html
│   └── application.properties          2.2

```

يمثل المخطط هيكل مشروع لتطبيق ويب مبني باستخدام Spring MVC , REST حيث يوضح كيف يقسم الكود الى طبقات:

(1) هو المجلد الذي يحتوي الكود البرمجي ويقسم إلى

1.1 الكلاس الأساسي لتشغيل التطبيق

2.1 كلاس جافا يمثل هيكل الكتاب في قاعدة البيانات

3.1 التواصل مع قاعدة البيانات

4.1 المنطق الخاص بالتطبيق

5.1 استقبال طلبات الويب وتوجيهها

(2) يحتوي على الملفات غير البرمجية

2.1 يحتوي ملفات HTML التي يتم عرضها للمستخدم في المتصفح

2.2 ملف الإعدادات الأساسي للتطبيق.

❖ فائدة أريسيزية

Entity هو الجدول الذي سيتم تخزينه في قاعدة البيانات ويستخدم Spring تقنية JPA لربط كود جافا بقواعد البيانات تلقائياً.

بمعنى آخر أي أنك ستكتب الجدول الذي تريد انشاؤه في قواعد البيانات (اسمه، ماذا يحتوي من صفوف) بلغة جافا و spring سيتكفل بإنشاء جدولك .

Example: Entity-Book.java

مثال 2:

```
package com.library.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
    public Book() {}
    public Book(String title, String
author) {
        this.title = title;
        this.author = author;
    }
    // Getters and Setters
    public String getTitle() { return
title; }
    public void setTitle(String title)
{ this.title = title; }
    .....
}
```

يوضح مثال 2 ما ذكرناه سابقاً حيث يعطينا مثال عن انشاء جدول باسم (book) حيث: @Entity: تخبر Spring هذا الكلاس يمثل جدول
 الكلاس اسمه book
 @Table(name = "books") : أي
 @Id : أي حقل id هو المفتاح الرئيسي (primary key) للجدول
 @GeneratedValue (strategy = GenerationType.IDENTITY)
 أي أنشئ قيمة id تلقائياً عند إضافة كتاب جديد .

مثال 3:

```
package com.library.repository;
import com.library.entity.Book;
import
org.springframework.data.jpa.repository
.JpaRepository;
import
org.springframework.stereotype.Reposito
ry;
@Repository
public interface BookRepository extends
JpaRepository<Book, Long> { }
```

Repository-BookRepository.java
 مسؤول عن عمليات التواصل مع قاعدة البيانات
 بمجرد كتابة هذه الأسطر يصبح لدينا وصول تلقائي لمجموعة ضخمة من الدوال الجاهزة للتواصل مع قاعدة البيانات منها:
 findById() Save()
 deleteById() findAll()

Book Service.java

مثال 4:

تعمل هذه الطبقة كوسيط بين المتحكم
controller و repository حيث تعالج
البيانات القادمة قبل إرسالها لقاعدة
البيانات
في الكود يقوم الكلاس باستدعاء الدوال
الجاهزة من Repository لتنفيذ
المهام:

```
package com.library.service;
import com.library.entity.Book;
import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class BookService {
    @Autowired
    private BookRepository bookRepository;
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }
    public Book getBookById(Long id) {
        return bookRepository.findById(id).orElse(null);
    }
    public Book saveBook(Book book) {
        return bookRepository.save(book);
    }
    public void deleteBook(Long id) {
        bookRepository.deleteById(id);
    }
}
```

Example: controller-Book Controller.java

مثال 5:

الوظيفة الأساسية لهذا الكلاس هي
استقبال الطلبات من المتصفح، التواصل
مع طبقة الخدمة لجلب البيانات، ثم تحديد
أي صفحة HTML يجب عرضها للمستخدم

```
package com.library.controller;
import com.library.entity.Book;
import com.library.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;
    @GetMapping("/{id}")
    public String viewBook(@PathVariable Long id, Model model) {
        Book book = bookService.getBookById(id);
        model.addAttribute("book", book);
        return "books/detail"; // this returns detail.html
    }
    .....
}
```

مثال 6:

صفحة HTML عادية مدعومة بأكواد thymeleaf وظيفتها استقبال البيانات التي أرسلها المتحكم (controller) عبر Model وعرضها بشكل مرئي للمستخدم في المتصفح

```
<!DOCTYPE html>
<html
xmlns:th="http://www.thymeleaf.org"
<head>
  <title th:text="${book.title} + ' -
Book Details' ">Book Details</title>
</head>
<body>
  <div>
    <h2
th:text="${book.title}">Title</h2>
    <p>Author: <span
th:text="${book.author}">Author</span>
    </p>
  </div>
</body>
</html>
```

مثال 7:

هذا الكلاس هو نقطة الانطلاق التي يبدأ منها تشغيل كل شيء في التطبيق حيث دالة main هي الدالة التي يبدأ منها تنفيذ البرنامج تقوم باستدعاء دالة SpringApplication.run() التي تقوم بتشغيل الحاوية (Spring Container) إنشاء الكائنات (Beans Creation) حقن التبعيات (Dependency injection) تجهيز خادم الويب المدمج

```
import com.library.entity.Book;
import com.library.repository.BookRepository;
import org.springframework.boot.CommandLineRun
ner;
import org.springframework.boot.SpringApplicat
ion;
import org.springframework.boot.autoconfigure.
SpringBootApplication;
import org.springframework.context.annotation.
Bean;

@SpringBootApplication
public class LibraryMvcApplication {
    public static void main(String[]
args) {

        SpringApplication.run(LibraryMvcApplic
ation.class, args);
    }
}
```

🔗 والآن لنستكمل ما تعلمناه في المحاضرة السابقة

في المحاضرة السابقة تعرفنا على أساسيات springboot وتعرفنا على مفاهيم Beans, DI, IoC, Annotation والآن سنتعلم بناء تطبيق ويب متكامل (MVC) و واجهة برمجة (REST API) جاهزة للإنتاج

ملاحظة:

استخدام Spring MVC يمثل طريقة منظمة للتعامل مع طلبات HTTP بدلاً من التعامل مع Raw Sockets

Spring MVC

■ MVC(Model-View-Controller)

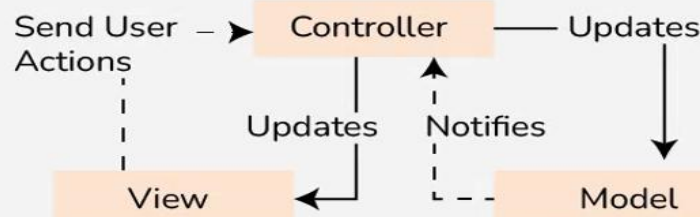
هو نمط هندسي للبرمجيات يفصل التطبيقات إلى ثلاث مكونات رئيسة لضمان تنظيم الكود وسهولة صيانه

■ الطبقات الثلاثة:

1. النموذج (Model) : يمثل طبقة البيانات ومنطق العمل (Business logic) هو المسؤول عن إدارة حالة البيانات والتعامل مع قواعد البيانات
2. واجهة العرض (view) : هي الطبقة التي يراها المستخدم (مثل صفحات HTML) مسؤوليتها عرض البيانات التي يوفرها النموذج (Model) بطريقة بصرية
3. المتحكم (controller) يعمل كواجهة استقبال لمدخلات المستخدم يقوم بمعالجة الطلبات والتواصل مع النموذج (Model) ثم يوجه واجهة العرض (View) لتحديث ما يراه المستخدم بناءً على التغييرات

مخطط 1 :

MVC Design Pattern



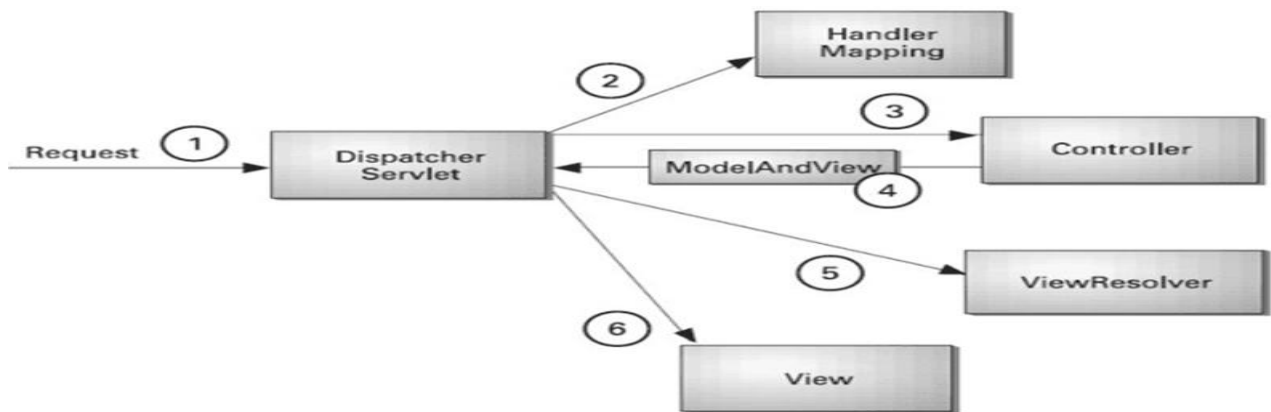
يوضح المخطط دورة حياة الطلب:

1. User Action : أي عندما يقوم المستخدم بفعل ما يرسل للمتحكم (مثل نقر على زر)
2. التحديث update : يقوم المتحكم بتحديث النموذج بناءً على طلب المستخدم
3. الإشعار (Notifies) : يقوم النموذج بإخبار واجهة العرض بوجود تحديثات

4. العرض: تقوم واجهة العرض باسترجاع البيانات من النموذج وعرضها للمستخدم

Spring MVC Architecture

تعتبر Spring MVC إطار عمل للويب مبني ضمن إطار عمل spring الأوسع وهو مخصص لبناء تطبيقات ويب تعتمد على لغة جافا
مخطط 2:



يوضح تسلسل خطوات معالجة الطلب:

1. استلام الطلب (request) : يرسل المستخدم طلباً (مثل (getbooks)) يستقبل هذا الطلب من قبل Dispatcher servlet (وهو المتحكم الأمامي المسؤول عن تنسيق كافة العمليات)
2. (Handler Mapping) : يقوم Dispatcher servlet بسؤال ال handlerMapping لمعرفة أي controller مسؤول عن معالجة الرابط تحديداً
3. يرسل الطلب إلى controller
4. يعالج ال controller الطلب ويعيد ModelAndView يحوي البيانات واسم الصفحة المراد عرضها (view)
5. يقوم Dispatcher servlet بإرسال اسم الصفحة (view) الى view Resolver ليقيم الأخير بتحديد ملف HTML الفعلي
6. عرض النتيجة النهائية (View) : تدمج البيانات مع ملف HTML وترسل الصفحة للعميل

Zero Configuration setup

```

<!-- pom.xml -->
<dependencies>
  <dependency>

<groupId>org.springframework.boot</grou
pId>
    <artifactId>spring-boot-
starter-web</artifactId>
    </dependency>
  <dependency>

<groupId>org.springframework.boot</grou
pId>
    <artifactId>spring-boot-
starter-thymeleaf</artifactId>
    </dependency>
</dependencies>

```

تمثل الأسطر السابقة إعدادات التكوين
الصفري (إعدادات تلقائية)
حيث تقوم بإضافة مكتبتين
أساسيتين (starters) :

1. Spring-boot-starter-web : تتضمن
كافة المكتبات اللازمة لبناء تطبيقات الويب
بما في ذلك Spring MVC و خادم
Tomcat

2. Spring-boot-starter-thymeleaf
إنشاء واجهات عرض HTML ديناميكية

What we get Automatically? (ما الذي يحصل عليه تلقائياً)

1. Embedded tomcat server يتم تشغيل خادم tomcat مدمج تلقائياً على المنفذ 8080
2. Dispatcher servlet mapped t “\” يتم ضبط التحكم الأمامي Dispatcher servlet ليتولى معالجة كافة الطلبات القادمة
3. Thymeleaf template resolution : يتم إعداد محرك Thymeleaf تلقائياً للبحث عن ملفات الواجهة داخل مجلد templates
4. static resource serving : يتم تفعيل خدمات الملفات الثابتة (مثل الصور وملفات CSS) تلقائياً من خلال مجلد static
5. Error page handling : يوفر التطبيق آلية تلقائية لعرض صفحات الخطأ في حال حدوث مشاكل



```
package com.library.controller;
import
org.springframework.stereotype.Controller;
import
org.springframework.web.bind.annotation
.GetMapping;
@Controller // Marks this as a web
controller
public class HomeController {
    // Handles GET requests to
    "/welcome"
    @GetMapping("/welcome")
    public String showWelcomePage() {
        return "welcome-page"; //
        Looks for welcome-page.html
    }
}
```

يمثل كلاس controller (شرحنا عمله سابقاً)

```
@Controller
public class ProductController {
    @GetMapping("/product-details")
    public String
    showProductDetails(Model model) {
        // Create data
        Product product = new
        Product("Laptop", 999.99);
        // Add to model (name, value)
        model.addAttribute("product",
        product);
        model.addAttribute("pageTitle",
        "Product Info");
        return "product-view"; //
        Template (product-view.html) gets the
        data!
    }
}
```

تمثل الأسطر السابقة Model والتي تستخدم لتمرير البيانات من controller لمواجهة العرض حيث يتم انشاء كائن بيانات جديد (1).
(2) نستخدم addAttribute لتخزين البيانات داخل الحاوية.

(3) return "product view"
يقيد الدالة نصاً "product view" وهو اسم ملف HTML الذي سيستقبل هذه البيانات ويعرضها

خصائص الكائن Model

- يقوم spring بتوفير الكائن model للمتحكم بشكل تلقائي
- تخزين بيانات: حيث يعمل بنظام (key. Value) حيث key الاسم لصفحة HTML و value البيانات الفعلية
- يتم الوصول للبيانات باستخدام key
- يمكن إضافة عدد غير محدد من البيانات إلى كائن model واحد قبل إرساله للواجهة

```
<!DOCTYPE html>
<html
xmlns:th="http://www.thymeleaf.org">
<head>
  <title th:text="${book.title} + ' -
Book Details' ">Book Details</title>
</head>
<body>
  <div>
    <h2
th:text="${book.title}">Title</h2>
    <p>Author: <span
th:text="${book.author}">Author</span>
</p>
  </div>
</body>
</html>
```

يمثل واجهة العرض والتي تستخدم th:text لعرض البيانات ديناميكياً.



Handling User Input

أي كيفية استقبال البيانات التي يكتبها المستخدم في روابط الويب ومعالجتها داخل المتحكم

* الرابط المستهدف هنا هو /search? Keyword=laptop

* @GetMapping("/search")

Get تحدد أنني أريد استقبال طلبات من نوع Get فقط

/search أي path الطلب يحتوي

=> إذا تحقق كل منهما سيوجه الطلب للدالة search products

* الدالة search products : نأخذ الدالة من خلال

@RequestParam String keyword القيمة المرتبطة بكلمة

keyword من الرابط (في المثال هي laptop) وتخزنها في

المتغير keyword

@RequestParam(defaultvalue="1") int page

تأخذ رقم الصفحة من الرابط وفي حال لم يرقم المستخدم بتحديد

رقم الصفحة تكون القيمة الافتراضية 1.

* المعالجة : يتم استدعاء خدمة البحث search Service.find وإضافة الناتج إلى Model لعرضها في صفحة

"search=results"

في هذا المثال تعرفنا على أهم أنواع البارامترات وهي @RequestParam ولاحظنا أنها تستخدم لجلب

البيانات من الروابط (URL query parameters)

@ModelAttribute : تستخدم لربط بيانات النماذج (form) مباشرة بكائنات جافا.

@ default value : تستخدم لتحديد قيم افتراضية للبارامترات.

Path Variables & Dynamic URLs

Dynamic URLs

هي الروابط التي تحتوي على أجزاء متغيرة تتغير بناء على البيانات التي يطلبها المستخدم بدلاً من وجود صفحة ثابتة لكل عنصر.

Path Variables

هي القيم الفعلية التي يتم وضعها داخل هذه الروابط لتحديد هوية المورد المطلوب

لدينا أسلوبيين لتصميم الروابط

1. الروابط التقليدية (Traditional): يستخدم علامة الاستفهام ? لتعريف البيانات مثل

/post?id=123

الروابط النظيفة (Clean MVC): يدمج القيمة مباشرة كجزء من مسار العنوان مثل

/post/123 وهنا يسمى 123 متغير وال path variable

لجعل الرابط ديناميكي :

(1) تحديد المسار بالأقواس ({}) : أي نضع أقواس داخل اسم مستعار للجزء المتغير داخل الرابط

مثل

@GetMapping ("post/{postId}")

(2) نستخدم @path variable : حيث يسحب القيمة من الرابط ونضعها في متغير لنتمكن من

استخدامه

يمكن أن يحتوي الرابط الديناميكي على أكثر من متغير

مثال :

/category/{catId}/post/{postId}

```
@Controller
public class BlogController {
    // Traditional: /post?id=123
    // Clean MVC: /post/123
    @GetMapping("/post/{postId}")
    public String showPost(
        @PathVariable Long postId, Model model)
    {
        Post post =
            postService.findById(postId);
        model.addAttribute("post",
            post);
        return "post-detail";
    }
    // Multiple path variables

    @GetMapping("/category/{catId}/post/{postId}")
    public String showCategorizedPost(
        @PathVariable String catId,
        @PathVariable Long postId)
    {
        // ...
    }
}
```

Complete Flow Example

مثال شامل يوضح لنا ما تم شرحه سابقاً ويلخص الرحلة في 5 خطوات أساسية:

1. المستخدم يطلب الرابط (نوع GET)

<https://localhost.8080/product/42>

42 هو ال path variable

2. DispatcherServlet receives request

الاستقبال:

- يقوم DispatcherServlet (المتحكم الأمامي) باستلام الطلب

- بناء على الإعدادات يدرك أن هذا الرابط يجب أن يوجه إلى كلاس product controller

3. معالجة المتحكم controller processes

- يوجه الرابط للدالة المناسبة

- تقوم الدالة باستدعاء خدمة لجلب المنتج
productservice.getByld(id)

- يوضع كائن للمنتج داخل ال Model

- تعيد الدالة اسم القالب product-details

4. المعالجة في Thymeleaf يقوم محرك القوالب بفتح ملف product-details.html

ويستبدل كلمة product بالاسم الحقيقي للمنتج رقم 42 (مثلاً : iPhone)

5. النتيجة النهائية (final HTML) إرسال صفحة HTML النهائية الى المتصفح

From Browser to HTML - Full Journey

1. User visits:

<http://localhost:8080/products/42>

└ GET /products/42

2. DispatcherServlet receives request

└ Routes to ProductController

3. Controller processes:

@GetMapping("/products/{id}")

public String

showProduct(@PathVariable Long id,

Model m) {

Product p =

productService.getById(id);

m.addAttribute("product", p);

return "product-detail";

}

4. Thymeleaf processes product-

detail.html:

<h1

th:text="\${product.name}">Product</h1>

5. Final HTML sent to browser!

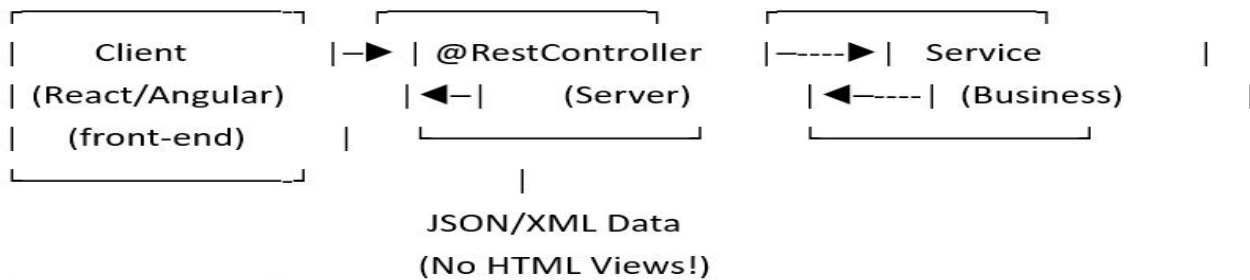


From MVC to REST API

- Evolution: HTML Views → JSON APIs
- Traditional MVC:



- Modern REST API :



Key Shift: Server returns data, not presentation

التحول من نظام MVC التقليدي إلى نظام REST API الحديث:

- (1) التطور من واجهات HTML إلى واجهات JSON : أي الانتقال من إرسال صفحات ويب كاملة للمتصفح إلى إرسال بيانات خام (غالباً بصيغة JSON)

MVC vs REST API

⚡ نظام MVC التقليدي :

في هذا النظام يكون الخادم مسؤول عن كل شيء حيث:

- (1) المتصفح: يطلب صفحة HTML
- (2) المتحكم controller : يستقبل الطلب ويعالج البيانات
- (3) واجهة العرض (HTML View -Thymeleaf) : يقوم الخادم بدمج البيانات داخل قالب HTML وإرسال الصفحة كاملة وجاهزة للمتصفح
- (4) النتيجة : المستخدم يستلم صفحة HTML لا يمكن لغير المتصفح فهمها بسهولة

نظام REST API الحديث (Modern REST API)

هنا يتغير دور الخادم ليصبح "مزود بيانات" فقط حيث:

- (1) العميل (client) : قد يكون متصفحاً يستخدم تقنيات حديثة (مثل React) أو تطبيق موبايل
- (2) المتحكم (REST Controller) : لا يعيد صفحات HTML بل يعيد بيانات فقط
- (3) طبقة الخدمة (Service) : تعالج منطق العمل وتمرر البيانات للمتحكم
- (4) النتيجة: يتم تبادل البيانات بصيغة JSON أو XML فقط دون أي واجهات عرض (No HTML View)

Key shift (التحول الجوهري) :

الخادم يعيد بيانات وليس شكلاً للعرض (Server returns data, not presentation)

REST principles

- (Stateless) عدم حفظ الحالة: كل طلب request مرسل من العميل يجب أن يحتوي على جميع المعلومات اللازمة لفهمه وتنفيذه ولا يعتمد الخادم على تذكر طلبات سابقة
- التوجه نحو الموارد (Resource-Based): كل شيء في النظام (كتاب - مستخدم - طلب) يعتبر "مورداً" وله عنوان URL فريد خاص به للوصول إليه
- HTTP Methods : GET (read), post(create), put(update), DELETE (remove)
- التمثيلات (Representations) : يتم تبادل البيانات بصيغ مثل XML , JSON ولا يتم استخدام لغة HTML

فائدة أريسيزية

ما هو ال Response Body @ :

يمكن القول بأنه مترجم للردود الناتجة عن الكود الخاص بك من لغة جافا إلى لغة البيانات العالمية (JSON) حيث وظيفته الأساسية هي إخبار Spring : لا تبحث عن صفحة HTML لعرضها بل خذ هذه البيانات وأرسلها مباشرة للمتصفح كما هي .

معادلة REST Controller

يمكن القول أن `@Rest controller = @controller+@ResponseBody` إذاً من المعادلة نستنتج أن ال `Rest controller` يدمج لنا وظيفتين معاً لنرى الفرق بين المثالين الآتيين لتوضح الصورة لنا أكثر

هذا الكود يوضح لنا لطريقة التقليدية:

-حيث نستخدم `@Controller` لتعريف الكلاس

-نضطر لإضافة `@ResponseBody` فوق كل دالة داخل الكلاس

=> هنا لن يبحث Spring عن صفحة HTML بل يرسل كائن المنتج (product) مباشرة للعميل بعد تحويله إلى JSON

```
// Traditional approach (verbose)
@Controller
public class OldApiController {
    @GetMapping("/product")
    @ResponseBody // <<< Needed on
every method!
    public Product getProduct() {
        return product; }
}
```

أما الكود التالي يوضح لنا الطريقة الحديثة

(Modern REST approach)

وسنلاحظ كيف يصبح الأمر أسهل مع `@Rest controller`

-نضع `Rest controller` مرة واحدة فقط فوق الكلاس

-النتيجة : تصبح جميع الدوال داخل الكلاس تحمل وسم `@Response Body` تلقائياً وضمناً

حيث يتم تحويل الكائنات إلى صيغة JSON آلياً دون الحاجة لتكرار الكود

```
// Modern REST approach (clean)
@RestController // <<< Combines
both!
public class ProductApiController {
    @GetMapping("/product")
    public Product getProduct() {
        return product; // Auto-converted to
JSON
    }
    // All methods automatically
    @ResponseBody
}
```

Spring Boot Auto configures

SpringBoot يقوم بثلاث مهام حيوية خلف الكواليس لتسهيل عمل ال APIs

- Jackson Json controller : المكتبة المسؤولة عن تحويل كائنات جافا إلى نصوص Json والعكس .
- Proper content-type headers : يقوم التطبيق تلقائياً بإضافة application/Json في رأس الاستجابة ليعرف العميل نوع البيانات المستلمة.
- Exception handling for APIs : يوفر معالجة منظمة للأخطاء تتناسب مع طبيعة ال APIs (مثل إرسال رسائل خطأ بصيغة Json بدلاً من صفحات HTML)

RESTful URL Design

قواعد تصميم الروابط

(1) Resources not Actions : وهو مبدأ جوهري حيث أن الروابط يجب أن تعبر عن الموارد (Resource) وليس عن الأفعال (Actions) أي في تصميم REST يجب أن يشير الرابط إلى اسم الشيء (مثل /product) بدلاً من فعل (مثل /get product / أو /create product) أما الفعل يتم تحديده برمجياً (GET , post) وليس عبر نص الرابط

(2) المسار الرئيسي (Base path) : مثل ("/api/product") @RequestMapping يستخدم فوق الكلاس لتحديد المسار الأساسي لجميع الدوال داخله، هذا يوفر تكرار كتابة الرابط في كل دالة ويجعل الكود أكثر تنظيماً

```
// GET /api/products
(Collection)
@GetMapping
public List<Product>
getAllProducts() { ... }
// GET /api/products/123
(Single resource)
@GetMapping("/{id}")
public Product
getProduct(@PathVariable Long id) { ... }
// POST /api/products
(Create new)
@PostMapping
public Product
createProduct(@RequestBody Product
product) { ... }
```

يستخدم المسار الرئيسي @GetMapping تعيد قائمة كاملة بكافة المنتجات ("/{id}") هنا يستخدم المسار الرئيسي ويدمج في نهايته رقم المتغير وذلك لجلب عنصر واحد محدد.

@PostMapping لإنشاء مورد جديد

REST principles

مفهوم REST المعياري : REST هو اختصار (Representational State Transfer) وهو ليس لغة برمجة بل نمط معماري يستخدم لتنظيم التواصل بين الأنظمة

🔑 القيود الستة لتصميم web APIs

- (1) معماري العميل والخادم (client-server Architecture) تعتمد على فصل المسؤوليات فالعميل يهتم بواجهة المستخدم بينما الخادم بتخزين وإدارة منطق البيانات .
- (2) عدم حفظ الحالة (Stateless) : كل طلب (request) عليه ان يحتوي على جميع المعلومات اللازمة لفهمه وتنفيذه.
- (3) القابلية للتخزين المؤقت (cacheable) الاستجابة القادمة من الخادم يجب ان تحدد إذا ما كانت البيانات قابلة للتخزين المؤقت أم لا.
- (4) واجهة موحدة (Uniform interface) يجب أن يكون الوصول للموارد منظم وموحد حيث كل مورد (مثل مستخدم أو طلب) يتم تعريفه برابط فريد (URL) مثل User/123 .
- (5) نظام الطبقات (layered system)
- (6) Code on demand الكود عند الطلب : قد يقوم الخادم أحياناً بإرسال كود تنفيذي .

RESTful Resources HTTP Methods

- Resources = Nouns (Not Actions) •
- /students (Collection of students) •
- /students/{id} (Single student) •
- /courses (Collection of courses) •
- /courses/{id} (Single course) •
- HTTP Methods = Verbs •
- GET → Retrieve resource(s) •
- POST → Create new resource •
- PUT → Update/replace resource •
- DELETE → Remove resource •
- combination creates a clean, predictable API •
- eg, GET /students/123 gets student 123 •

REST API Design Best practices

أفضل الممارسات لتصميم واجهات برمجة تطبيقات

➤ ممارسات ينصح بها (Do)

- (1) استخدام أسماء للمجموعات (use plural nouns for collections) مثل /users بدلاً من users
- (2) إرجاع رموز حالة HTTP أي رمز يعبر عن حالة الطلب (مثل 200)
- (3) استخدام الموارد المتداخلة للعلاقات: مثال الوصول لطلبات مستخدم معين يكون عبر /users/{id}/orders
- (4) تحديد إصدار ال API الخاص بك

➤ ممارسات يجب تجنبها (Don't)

- (1) استخدام الأفعال في الروابط (كما وضعنا سابقاً)
- (2) إرجاع صفحات خطأ: أي يجب إرسال تفاصيل الخطأ بصيغة Json وليس صفحات HTML
- (3) خلط التسمية بين المفرد والجمع

Building REST controller

➤ الأساس البرمجي لبناء المتحكمات

- (1) المكونات الأساسية
@Rest controller نبدأ به أي كلاس
@RequestMapping("/api/books") يستخدم لتحديد المسار الأساسي بعدها نعرف الكلاس

Basic CRUD Endpoints

➤ إضافة أريسيزية

CRUD هي اختصار (create, read, update, Delete)

يوضح المثال التالي كيفية إدارة العمليات CRUD

حيث :

1) القراءة (read) : الرابط GET/api/books يمثل
الوظيفة: جلب قائمة بكل الكتب المتاحة
أما GET/api/book/123 يمثل قراءة عنصر واحد

create : الرابط post/api/books

التحديث update : الرابط put/api/book/123

```
@RestController
@RequestMapping("/api/books")
public class BookController {
    @GetMapping // GET /api/books
    public List<Book> getAllBooks() {
    ... }
    @GetMapping("/{id}") // GET
    /api/books/123
    public Book getBook(@PathVariable
    Long id) { ... }
    @PostMapping // POST /api/books
    public Book
    createBook(@RequestBody Book book) {
    ... }
    @PutMapping("/{id}") // PUT
    /api/books/123
    public Book
    updateBook(@PathVariable Long id,
    ...
    }
}
```

Path variables & request parameters

■ Pathvariable : متغيرات المسار عندما يكون لدينا مسار يحوي متغيرات متعددة مثل :
@GetMapping("users/{userId}/orders/{orderId}") حيث لدينا متغيرات هنا userId وهو
معرف المستخدم orderId وهو معرف الطلب
يتم ربط كل متغير في الرابط ب parameter مستقل مثل

```
Public order getOrder
(@pathvariable Long userId,
@pathvariable Long orderId)
```

Request parameter : هي لاستقبال البيانات من الروابط ونقلها للخادم عبر سلسلة استعلامات
(?) تستخدم عادة لعمليات البحث او تقسيم الصفحات

```
Request Parameters (@RequestParam):
@GetMapping("/search")
public List<User>
searchUsers(@RequestParam String
name,

@RequestParam(required = false,
defaultValue = "1") int page) {
    // /search?name=John&page=2
}
```

وهنا name String @RequestParam يسحب القيمة المرتبطة ب name من الرابط

أما @RequestParam(required=false,defaultValue="1") أي المتغير page Required=false ليس من الضروري أن يرسل default value فإن لم يرسل يكون تلقائياً 1.

Receiving JSON/XML in Requests

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    // Spring automatically converts JSON → User object
    return userService.save(user);
}
```

Request Body Example (JSON):

```
{
    "name": "John Doe",
    "email": "john@example.com",
    "age": 25
}
```

HTTP status codes

حالات الطلب

(1) النجاح Success-2xx نجاح عام 200

نجاح إنشاء مورد 201 وغالباً يتضمن location header
الطلب قد نجح ولكن 204 لا يوجد بيانات لإعادةتها للعميل

فائدة أريسيزية

200 يستخدم غالباً مع GET و Put

(2) الفشل (خطأ من العميل) client Error codes-4xx

400 Bad request يحوي بيانات غير صالحة أو الطلب غير مفهوم

401 unauthorized أي العملية تطلب تسجيل الدخول

403 for bidden العميل لا يملك صلاحيات الوصول

404 Not found المورد المطلوب غير موجود

(3) رموز خطأ الخادم (Server Error codes)

خطأ داخلي 500 (في كود جافا أو في التواصل مع قاعدة البيانات).

The End

